Alpha Fitness Manual técnico

Edición: 2.0 · 19 de octubre de 2026.

Revisión: base `06d31c2984133a9596bff0db21935399eb8c658c` más cambios locales de ventas simuladas, progreso, navegación móvil y cierre documental del 19 de octubre. Sus huellas se identifican en REVISION.json. Esto no equivale a un nuevo commit publicado.

Dirigido a: desarrollo, mantenimiento, administración del sistema y evaluación técnica.

Este documento explica la implementación actual. La instalación, actualización y recuperación están desarrolladas en Operación y despliegue; las columnas y restricciones se detallan en el Diccionario de datos. El Manual de usuario explica las tareas desde la interfaz.

1. Propósito y límites

Alpha Fitness es una aplicación web de gestión de gimnasio: personal y clientes, membresías, asistencia, catálogo y ventas, rutinas, sesiones, progreso y reportes financieros. Las pantallas se adaptan al teléfono y ofrecen navegación inferior por rol. No existe una aplicación móvil nativa ni una API pública versionada en esta revisión.

Las nuevas ventas realizadas desde la interfaz usan dinero simulado. Una venta aprobada crea registros y descuenta inventario, pero no mueve dinero ni contacta bancos. La tarjeta virtual permite elegir aprobación o rechazo y no solicita números de tarjeta, CVV ni vencimiento. El backend conserva el registro manual sin bandera de simulación para compatibilidad; tampoco representa una integración bancaria. Los pagos de membresía son registros manuales confirmados por Secretaria y se consideran ingresos registrados en reportes.

Los rangos de progreso reconocen cantidad de registros; no evalúan fuerza, técnica o salud. Los reportes presentan ingresos, sin costos, gastos, utilidad, impuestos, conciliación bancaria ni devoluciones automatizadas.

2. Pila y requisitos

Las versiones se toman de los lockfiles, no de la última versión publicada en Internet.

Componente	Versión o requisito	Uso
PHP	`^8.2`; comprobado localmente con 8.2.12	Backend y Artisan
Laravel	12.69.3	Aplicación, sesiones, validación, ORM y pruebas
Laravel Socialite	5.29.0	Acceso opcional de clientes con Google
Laravel DomPDF / DomPDF	3.1.2 / 3.1.6	Reportes PDF
PhpSpreadsheet	5.10.0	Reportes Excel

Componente	Versión o requisito	Uso
Vite / Laravel Vite Plugin	7.3.6 / 2.1.0	Compilación e integración de assets
Tailwind CSS	4.3.3	Estilos junto con CSS propio
PHPUnit	11.5.56	Pruebas backend
Puppeteer Core	25.13.0	Revisión con navegador instalado
Node.js	`^20.19.0` o `>=22.12.0`, según Vite fijado	Compilación y herramientas frontend
Composer	Compatible con los requisitos del lockfile	Dependencias PHP

Se necesitan PDO y driver del motor elegido, mbstring, OpenSSL, ctype, filter, hash, session, tokenizer, DOM/XML, fileinfo, iconv, simplexml, xmlreader, xmlwriter, ZIP, zlib y GD con WebP; cURL y certificados válidos para HTTPS. Confirmar el conjunto efectivo mediante `composer check-platform-reqs`. No omitir requisitos de plataforma para declarar operativo un servidor.

El entorno verificado utiliza Windows, PHP de XAMPP y SQLite. Otros motores tienen configuración disponible; esta entrega no certifica concurrencia ni operación en MySQL, MariaDB, PostgreSQL o SQL Server. XAMPP no es un requisito del producto.

3. Arquitectura y organización

Es un monolito Laravel: solicitud → `public/index.php` → middleware y autorización → controlador → servicios/modelos/base → Blade, redirección, JSON o descarga. Artisan arranca la misma aplicación para tareas CLI.

Ubicación	Responsabilidad
`bootstrap/app.php`	Arranque, salud `/up`, middleware y campos sensibles que no se reflashean
`routes/web.php`	Rutas, verbos, nombres, gates y límites
`app/Http/Controllers`	Validación, permisos de recurso y coordinación
`app/Models`	Relaciones Eloquent, casts, estados y protección del administrador
`app/Services/RegistroAsistencia.php`	Entrada, salida y cierre de visitas antiguas
`app/Services/ClientesEntrenador.php`	Vínculo mediante rutinas asignadas
`app/Services/PagoSimulado.php`	Efectivo, cambio y tarjeta de prueba
`app/Services/ProgresoCliente.php`	Resumen, filtros, mejores series y gráfico

Ubicación	Responsabilidad
app/Services/AnaliticaFinancieraService.php	Indicadores y exclusión de simulaciones
app/Services/ReportePdfService.php, ReporteExcelService.php	Exportaciones
app/Services/ImagenSegura.php, Invitaciones.php	Archivos recodificados e invitaciones
app/Support	Acceso, apariencia, dinero en centavos y rangos
resources/views, resources/css, resources/js	Interfaz Blade y entradas Vite
database/migrations, database/seeders	Esquema evolutivo y catálogos iniciales
tests, tools	Pruebas y utilidades de verificación

public/build/manifest.json vincula entradas con archivos compilados que incluyen hash. Código y assets deben corresponder a la misma revisión. Una compilación correcta por sí sola no demuestra el comportamiento del servidor realmente publicado.

4. Identidades, roles y permisos

Hay dos guards de sesión: web usa users para personal y cliente usa clientes. Un empleado y un cliente pueden compartir número de ID; rutinas, votos de ejercicios y sesiones diferencian user_id y user_type (web o cliente). Los logins cierran el guard contrario y regeneran sesión. Controller::actual prioriza cliente si coexistieran ambos.

CuentaActiva controla actividad y contraseña establecida del personal en cada solicitud web. auth.session comprueba continuidad del hash; cambiar/restablecer contraseña rota el token de recuerdo. La siguiente tabla expresa gates de Acceso; también se aplican propiedad y vínculo con el recurso.

Gate	Admin	Secretaria	Entrenador	Cliente
administrar	Sí	No	No	No
inventario	Sí	Sí	No	No
existencias	Sí	No	No	No
membresias	Sí	Sí	No	Sí
asistencia	Sí	Sí	No	No
analitica_financiera	Sí	No	No	No

Gate	Admin	Secretaria	Entrenador	Cliente
`asignar_rutinas`	Sí	No	Sí	No
`operaciones`	No	Sí	No	No
`progreso`	No	No	No	Sí

Administrador administra personal, clientes, planes, catálogo, existencias y reportes. Secretaria opera asistencia, ventas y activación/cancelación/aprobación de membresías. Admin no hereda el gate `operaciones`. Entrenador consulta clientes vinculados y su progreso; Cliente crea/elimina marcas propias. Roles desconocidos fallan de forma cerrada.

El administrador único utiliza `admin_key='unico'`, índice único y eventos del modelo. No puede desactivarse, cambiar de rol ni eliminarse mediante ese modelo. El alta de personal solo admite Secretaria y Entrenador, genera secreto aleatorio y deja `password_establecida=false` hasta usar el enlace privado. No hay contraseña inicial compartida. Los brokers de recuperación de personal y clientes usan tablas separadas, caducidad de 60 minutos y throttle de regeneración de 60 segundos.

5. Mapa de rutas y contratos

Las rutas usan sesión y CSRF para escrituras. El inventario exacto se obtiene con `php artisan route:list`. La tabla agrupa rutas internas, no una API externa.

Ruta o grupo	Función	Acceso efectivo
`/login`, `/cliente/login`, `/cliente/registro`	Acceso y alta de cliente	Público; POST 5/min
`/olvide-mi-contrasena`, `/establecer-contrasena`	Recuperación/enlace privado	Público; POST 5/min
`/cliente/google`, `/cliente/google/callback`	OAuth opcional	Público; 20/min, estado e identidad comprobados
`/informacion`, `/privacidad`, `/up`	Información, política y salud	Público
`/dashboard`, `/cliente/dashboard`	Inicio por rol	Guard correspondiente y sesión válida
`/configuracion` y subrutas	Perfil, clave, apariencia, avatar y Google	Cuenta autenticada y controles por operación
`/cuentas` y PATCH de ban/restauración	Actividad de clientes	Administrador
`/entrenadores` GET/POST/PUT	Directorio y personal	Lectura autenticada; alta/edición Admin

Ruta o grupo	Función	Acceso efectivo
`/entrenadores/{entrenador}/invitacion` POST	Reenvío de invitación pendiente	Admin; 5/min y throttle del broker
`/entrenadores/{entrenador}/calificar` POST	Valoración de 1-5 estrellas	Cliente vinculado y entrenador activo
`/entrenador/clientes` y detalle	Clientes/rutinas asignadas	Admin o entrenador vinculado
`/membresias` y `/membresias/planes`	Consulta y planes	Gate membresias; planes solo Admin
`/membresias/solicitudes` y pausas	Solicitar, activar, cancelar, pausar	Propietario o personal según ruta; operaciones solo Secretaria
`/asistencia` y subrutas	Entradas, salidas, cierre y CSV	Admin y Secretaria
`/mi-asistencia`	Historial propio	Cliente
`/productos`	Catálogo, alta y edición	Lectura autenticada; escritura inventario, stock existencias
`/ventas`, comprobante y correo	Simulación, historial y recibo	Admin y Secretaria
`/analitica`, `/analitica/pdf`, `/analitica/excel`	Consulta y exportación	Admin
`/entrenamientos` y subrutas	Crear, editar, asignar y finalizar	Cuenta autenticada; propiedad y asignación en controlador
`/ejercicios` y votos	Catálogo y calificación	Lectura/voto autenticado; gestión Admin
`/progreso` GET/POST/DELETE	Marcas propias	Cliente propietario
`/entrenador/clientes/{cliente}/progreso`	Consulta del progreso	Admin o entrenador vinculado
`/rankings`, `/estadisticas`	Enlaces anteriores	Redirigen a progreso del cliente

Los límites nombrados son `exportaciones` (10/min) y `operaciones-financieras` (60/min), por clase de identidad e ID, o IP cuando no hay usuario. No son cuotas comerciales.

Contratos JSON consumidos por la interfaz

Enviar `Accept: application/json`, contenido adecuado y token CSRF. Normalmente: validación 422 con `message` y `errors`; falta de sesión JSON 401; acceso ajeno 403; recurso inexistente 404; CSRF vencido 419; throttle 429. Formularios HTML usan redirecciones con errores/mensajes. El frontend confirma la respuesta antes de mostrar éxito.

Operación	Entrada relevante	Éxito JSON
Actualizar rutina	Nombre ≤100, objetivo ≤60, nivel permitido, días 1-7	`ok` y rutina
Crear/renombrar día	Rutina propia; título ≤100 al renombrar	`dia`
Buscar ejercicios	`q` y `grupo` opcionales ≤100	`ejercicios` con identidad, grupo y URLs
Añadir ejercicio	Ejercicio activo y día propio	`rutina_ejercicio`
Actualizar ejercicio	Series 1-20, repeticiones texto ≤20, peso nullable 0-999, descanso 0-600	Ejercicio actualizado
Reordenar	Todos los IDs propios del día sin repetidos	`ok:true`
Finalizar sesión	UUID, día propio, duración 0-86400, series/total 0-500, notas ≤500	`ok`, mensaje y sesión
Calificar ejercicio/entrenador	Estrellas enteras 1-5 y autorización	Estrellas, promedio y votos
Guardar apariencia	Diseño, modo y colores hex permitidos; contraste válido	Apariencia normalizada

El constructor serializa la cabecera, espera 600 ms tras cambios y avisa al salir con escritura pendiente. No es edición colaborativa. El avance del entrenamiento permanece en la página y puede perderse al recargar; las sesiones finalizadas se guardan en la base. El navegador conserva preferencias de apariencia/sonido según función; la apariencia de cuenta también puede persistirse en servidor.

6. Reglas de negocio e integridad

Ventas y simulación

`VentaController::store` exige UUID válido y lo normaliza a minúsculas. Un reintento del mismo vendedor devuelve la venta existente sin descontar nuevamente; un UUID ajeno responde 403. UUID distintos son operaciones distintas aunque el carrito coincida.

Admite hasta 100 líneas, agrupa productos repetidos y limita cada cantidad acumulada a 999. Calcula precios, subtotales y total en centavos con `Dinero`; no acepta el total del navegador. Bloquea vendedor, cliente opcional y productos ordenados por ID, comprueba actividad/stock y resuelve el pago antes de

crear venta, detalles y descuento en una transacción. El total máximo es USD 99,999,999.99.

La interfaz envía `pago_simulado=1`. Efectivo exige importe suficiente y hasta dos decimales; recibido y cambio se calculan en servidor. Tarjeta rechazada devuelve validación sin crear venta ni consumir stock; después admite aprobación con el mismo UUID. Transferencia guarda clasificación de prueba. La referencia es `SIM-` más UUID en mayúsculas. Campos de tarjeta/CVV están prohibidos y no se reflashean tras error.

Historial y comprobantes identifican la simulación. Métricas, comparativos, series, productos y métodos excluyen `pago_simulado=true`. La migración agrega default false y conserva el historial existente como no simulado; eso no acredita liquidación bancaria de esas filas.

Membresías y pausas

Solicitar copia nombre, precio, duración y condiciones del plan activo; editar el plan no modifica esa copia. La activación por Secretaria bloquea cliente y solicitud, crea membresía/pago y resuelve solicitud en transacción. Índices únicos evitan pagos duplicados. El importe confirmado admite hasta dos decimales, mínimo 0.01 y máximo 999999; no se exige igualdad con el precio acordado. No hay cobro electrónico.

La vigencia incluye extremos: `fin = inicio + duración − 1 día`. La renovación comienza después del último fin no cancelado vigente/futuro. Cancelar conserva pago e historial, marca cancelada y resuelve pausas pendientes/aprobadas; no devuelve dinero automáticamente.

Pausas: 3-30 días, motivo ≤255, inicio dentro de vigencia y máximo acumulado de 30 días por membresía. No coexiste otra pendiente/aprobada vigente o programada. La aprobación vuelve a validar con bloqueo; extender el fin no puede ocupar otra membresía no cancelada. Reanudar quita extensión no utilizada. El estado visual depende de fechas; no hay tarea propia programada que finalice automáticamente toda pausa aprobada. Los gates de ruta prevalecen sobre condiciones más amplias de un método interno.

Asistencia

Entrada exige cliente activo y rechaza pausa vigente o visita abierta; no exige membresía vigente. La salida puede cerrar visita aunque luego se desactive el cliente. Al intentar nueva entrada, una visita abierta de más de 12 horas se cierra con salida estimada entrada+2 horas; cierre masivo aplica igual regla. No es una hora observada.

El inicio calcula aforo con abiertas de últimas 12 horas, capacidad fija 80 y umbrales 50%/85%; asistencia puede contar todas las abiertas. El aforo no bloquea entradas. El cliente consulta historial y el personal registra movimientos.

Entrenadores, rutinas y sesiones

Asignar una rutina copia días/ejercicios para el cliente y guarda `asignado_por` histórico e ID estable. `asignado_por_id` determina el vínculo, no el nombre. El cliente ve sus entrenadores activos independientemente de búsqueda/paginación. Entrenador ve clientes/progreso vinculados; Admin puede consultar y filtrar responsables. ID nulo antiguo no otorga acceso por coincidencia de nombre. No hay campo único de «entrenador principal» en clientes.

Finalizar usa `sesion_uuid` para deduplicar reintentos; comprueba identidad, rutina y día. Series completadas no superan total. Tiempos/checklists proceden del navegador. Eliminar rutina conserva

sesiones con referencias nulas y nombres históricos. Abrir otra sesión genera una identidad nueva.

Progreso y rangos

Cliente registra ejercicio activo, peso 0.5-999 kg, repeticiones 1-100 y notas ≤255; solo él elimina sus marcas. Entrenador y Admin leen según vínculo. Los campos de verificación existen, pero no hay endpoint para certificar marcas desde el entrenador.

El gráfico usa hasta ocho registros cronológicos del mismo ejercicio: seleccionado o el del registro más reciente. Mejores series se eligen por peso × repeticiones para cada ejercicio, con desempate por ID reciente; no comparan fuerza entre ejercicios. El volumen corresponde a registros guardados. Ejercicios históricos inactivos siguen en filtros, pero no en altas.

Rango	Registros mínimos de toda la cuenta
Iniciado	0
Intermedio	3
Avanzado	8
Élite	15

Rango global independiente de filtros/paginación; borrar registros puede reducirlo. Las cuatro insignias propias están en `public/images/rangos`. La etapa muestra cuánto falta al siguiente umbral.

Analítica

Solo Admin consulta/exporta. Ingresos: `pagos_membresia.pagado_en` y ventas no simuladas por `created_at`; inicio ya no sustituye pagos ausentes por importes de membresías. Hay períodos diarios, semanales, mensuales y rango personalizado de hasta 90 días inclusivos. Un período abierto compara igual duración/hora de corte del anterior. Base anterior cero impide calcular crecimiento cuando el actual es positivo.

Métodos describen ventas; membresías se agrupan por origen porque su pago no guarda método. PDF/Excel comparten metodología. El log de exportación se crea antes del archivo: acredita intento, no entrega exitosa.

7. Seguridad, privacidad y archivos

Contraseñas con hash y mínimo 12 caracteres, mayúsculas/minúsculas, números y símbolos. No se observan MFA ni verificación obligatoria de correo local. Responsable y contacto de privacidad requieren datos institucionales antes de publicar.

Google comprueba estado, subject, correo y verificación del correo. No vincula una cuenta local solo por dirección coincidente. Vincular desde ajustes exige contraseña e identidad confirmadas durante cinco minutos. No guarda tokens access/refresh ni descarga avatar externo. Logout de Alpha Fitness no cierra Google del dispositivo.

Encabezados: nosniff, DENY, política de referencia, restricciones de dispositivos, COOP/CORP y CSP. La CSP actual admite scripts/estilos inline. HSTS se agrega en producción cuando Laravel reconoce HTTPS. Páginas de sesión usan `private,no-store`; reset usa `no-referrer`. Hay que comprobar HTTPS/proxy en el destino.

Las imágenes JPEG/PNG/WebP se decodifican, limitan a 16,777,216 píxeles y recodifican a WebP con UUID: hasta 640 px en avatar y 1200 en otras imágenes. Los controladores añaden límites por módulo. Se retira archivo nuevo si falla persistencia y anterior tras reemplazo. `public/images` es público por URL; no sirve para documentos privados.

CSV de asistencia incluye datos personales y protección frente a fórmulas. Excel fija texto para nombres y genera cinco hojas; PDF usa DomPDF sin recursos remotos. Bases, respaldos, sesiones, logs y enlaces de recuperación son privados. Correo `log/array` no entrega mensajes y la interfaz lo informa. No compartir logs completos para reenviar invitaciones.

8. Interfaz móvil

Navegación inferior por rol: Admin Inicio/Ventas/Clientes/Reportes; Secretaria Inicio/Ventas/Asistencia/Membresías; Entrenador Inicio/Mis clientes/Rutinas/Ejercicios; Cliente Inicio/Rutinas/Progreso/Mi plan. El menú lateral conserva funciones adicionales.

En menos de 768 px se reserva espacio inferior con safe area; controles relevantes tienen objetivo mínimo de 44 px y campos de contenido 16 px. Ventas/membresías convierten historial en tarjetas; progreso usa tarjetas. El cobro deja de ser fijo en móvil para evitar superposición. Mantener estos criterios al añadir acciones, errores, tablas o modales; probar teclado, foco, vacíos y textos largos.

9. Verificación y límites

La ejecución de cierre tras los últimos cambios de vista pasó 234 pruebas PHPUnit con 1842 aserciones; también pasaron los ocho grupos frontend. La revisión funcional incluye build Vite y navegador con cuatro roles. Los comandos, resultados y límites están en Verificación y pruebas; leer fuentes no equivale a ejecutar pruebas.

Se comprobaron efectivo/cambio, rechazo/reintento sin duplicado, exclusión de simulaciones, privacidad de progreso, insignias, membresías, permisos y pantallas móviles. La migración de simulaciones se aplicó a SQLite local tras respaldo coherente y conservación verificada de ventas originales.

No se ejecutaron cobros bancarios, SMTP real, OAuth productivo, restauración de un servidor productivo ni iOS físico. Chrome móvil emulado y SQLite no certifican todo navegador ni concurrencia del motor destino. Los avisos de correo, privacidad y Google de Admin requieren configuración válida.

10. Mantenimiento documental

Fuentes de comportamiento: rutas, `Acceso`, controladores, modelos, servicios, migraciones ordenadas y pruebas. Contrastar productor, consumidor y permisos cuando cambie un flujo; actualizar diccionario si cambia esquema y operación si cambian comandos/configuración.

Referencia complementaria: Despliegue oficial de Laravel 12.